Java Programming Assignment Answers - Part 2

1. Explain the Object class and discuss common methods inherited by Java classes.

The java.lang.Object Class:

In Java, every class is a direct or indirect subclass of the java.lang.Object class. If a class does not explicitly extend another class, it implicitly extends Object. Because Object is the root of the class hierarchy, its methods are inherited by all Java classes, providing a universal foundation for object behavior.

Common Methods Inherited from Object:

1. toString(): Returns a string representation of the object. By default, it returns a string consisting of the name of the class, the at-sign character `@`, and the unsigned hexadecimal representation of the hash code (e.g., ClassName@1a2b3c). Classes typically override this to provide meaningful state information.
2. equals(Object obj): Compares two objects for equality. By default, it uses reference equality (equivalent to `==`), checking if both references point to the exact same memory location. Classes frequently override this (along with hashCode()) to test for logical equality based on field values.
3. hashCode(): Returns an integer hash code value for the object. The general contract states that if two objects are equal according to equals(), they must produce the same hash code, making it critical for hash-based collections like HashMap and HashSet.
4. getClass(): Returns the runtime class descriptor (Class<?> object) of the object, which is used extensively in reflection.
5. finalize(): Called by the garbage collector on an object when garbage collection determines that there are no more references to the object (Note: deprecated in modern Java versions in favor of cleaner resource management like try-with-resources).
6. clone(): Creates and returns a shallow copy of the object (requires implementing the Cloneable marker interface).
7. Thread Synchronization Methods (wait(), notify(), notifyAll()): Facilitate inter-thread communication and synchronization.

2. Explain package member access for public, protected, default and private members with a suitable example.

Access Modifiers in Java:

Java provides four access control levels to regulate visibility of classes, methods, and variables:

• private: Accessible only within the defined class itself.
• default (package-private, no modifier): Accessible only within classes in the same package.
• protected: Accessible within the same package, and subclassed classes

residing in other packages.
• public: Accessible from anywhere across any package.

Code Example Illustrating Package Access:

```
package banking;

public class Account {
    private double balance = 1000.0;    // Accessible ONLY within Account class
    String accountNumber = "ACC123";  // default (package-private): accessible within 'banking' package
    protected String branchCode = "BR01"; // protected: package + subclasses in other packages
    public String bankName = "Global Bank"; // public: globally accessible

    public void showDetails() {
        // All members are accessible here inside the same class
        System.out.println(balance);
    }
}

// Subclass in the SAME package can access default, protected, and public members, but NOT private.
class SavingsAccount extends Account {
    void display() {
        // System.out.println(balance); // ERROR: private member not accessible
        System.out.println(accountNumber); // OK (same package)
        System.out.println(branchCode);    // OK (protected)
        System.out.println(bankName);      // OK (public)
    }
}
```

3. Create a multilevel inheritance hierarchy and demonstrate constructor execution order using super.

Multilevel Inheritance & Constructor Execution Order:

When a class inherits from another, and that class in turn has its own superclass, it forms a multilevel inheritance chain. During object instantiation of the lowest derived class, constructors execute in a top-down chain from the root parent down to the child class. The compiler implicitly or explicitly (via super()) invokes the parent constructor first before executing the child constructor body.

Java Implementation:

```
class Grandparent {
    public Grandparent() {
        System.out.println("1. Grandparent constructor executed.");
    }
}
```

```
class Parent extends Grandparent {
    public Parent() {
        super(); // Implicitly added if omitted; calls Grandparent constructor
        System.out.println("2. Parent constructor executed.");
    }
}

class Child extends Parent {
    public Child() {
        super(); // Calls Parent constructor
        System.out.println("3. Child constructor executed.");
    }
}

public class TestMultilevel {
    public static void main(String[] args) {
        Child obj = new Child();
        /* Output sequence:
           1. Grandparent constructor executed.
           2. Parent constructor executed.
           3. Child constructor executed.
        */
    }
}
```

4. Create a user-defined InvalidAgeException and use it to validate registration.

Custom Checked Exception Implementation:

```
// User-defined checked exception extending Exception
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

class UserRegistration {
    public static void registerUser(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Registration failed: Age must be at least
18. Given age: " + age);
        }
        System.out.println("Registration successful for age: " + age);
    }

    public static void main(String[] args) {
```

```
        try {
            registerUser(16); // Will trigger custom exception
        } catch (InvalidAgeException e) {
            System.out.println("Caught Exception: " + e.getMessage());
        }
        try {
            registerUser(21); // Valid registration
        } catch (InvalidAgeException e) {
            System.out.println("Caught Exception: " + e.getMessage());
        }
    }
}
```

5. Develop an integrated program combining inheritance, an interface, a custom exception and runtime polymorphism.

Integrated Java Application:

```
// 1. Custom Exception
class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String msg) {
        super(msg);
    }
}

// 2. Interface
interface Auditable {
    void generateAuditReport();
}

// 3. Base Class (Inheritance)
abstract class BankAccount implements Auditable {
    protected String accountNumber;
    protected double balance;

    public BankAccount(String accountNumber, double balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }

    public abstract void withdraw(double amount) throws
InsufficientFundsException;
}

// 4. Derived Class
class SavingsAccount extends BankAccount {
    private double interestRate;
```

```
    public SavingsAccount(String accountNumber, double balance, double
interestRate) {
        super(accountNumber, balance);
        this.interestRate = interestRate;
    }

    @Override
    public void withdraw(double amount) throws InsufficientFundsException {
        if (amount > balance) {
            throw new InsufficientFundsException("Savings Account overdraft
attempted for account: " + accountNumber);
        }
        balance -= amount;
        System.out.println("Successfully withdrew $" + amount + ". Remaining
balance: $" + balance);
    }

    @Override
    public void generateAuditReport() {
        System.out.println("Auditing Savings Account [ID: " + accountNumber + "],
Balance: $" + balance + ", Interest Rate: " + interestRate + "%");
    }
}

// 5. Driver Program demonstrating Runtime Polymorphism
public class BankSystem {
    public static void main(String[] args) {
        // Runtime Polymorphism: Parent reference pointing to Child object
        BankAccount acc = new SavingsAccount(
            "SAV-999",
            500.0,
            3.5
        );

        acc.generateAuditReport();

        try {
            acc.withdraw(200.0); // Valid withdrawal
            acc.withdraw(400.0); // Triggers custom exception
        } catch (InsufficientFundsException e) {
            System.out.println("Transaction Error: " + e.getMessage());
        }
    }
}
```